

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ № 1-5
по дисциплине Теория графов

«Алгоритмы на графах»
Вариант 1

Студент,
группы 5130201/40003

_____ Адиатуллин Т.Р

Доцент

_____ Востров А.В

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Математическое описание	6
1.1 Граф и его определение	6
1.2 Распределение классического Вейбулла	6
1.3 Генерация связного ациклического графа по степеням вершин	7
1.4 Эксцентриситет, центр и диаметральные вершины	8
1.5 Метод Шимбелла	8
1.6 Упорядочивание графа по алгоритму Фалкерсона	9
1.7 Нахождение кратчайшего пути. Алгоритм Флойда-Уоршалла	9
1.8 Определение сети	9
1.9 Поток в сети	9
1.10 Алгоритм Форда–Фалкерсона	10
1.11 Нахождение заданного потока минимальной стоимости	10
1.12 Теорема Кирхгофа	11
1.13 Алгоритм Краскала	11
1.14 Код Прюфера	12
1.15 Максимальное независимое множество вершин	12
1.16 Эйлеровы графы	13
1.17 Достройка до эйлерова графа	13
1.18 Алгоритм построения эйлерова цикла	14
1.19 Фундаментальная система циклов и циклический ранг	14
1.20 Симметрическая разность циклов	15
2 Особенности реализации	16
2.1 Модуль common	16
2.1.1 Graph<T>: хранение графа	16
2.1.2 WeibullDistribution: генерация случайных чисел	17
2.1.3 Generator<T>: генерация графов	17
2.2 Модуль lab1	18
2.2.1 Эксцентриситеты, центр и диаметральные вершины	18
2.2.2 Shimbelsolver<T>: метод Шимбелла	20
2.2.3 Поиск всех маршрутов (DFS с возвратом)	20
2.3 Модуль lab2	21
2.3.1 Fulkerson<T>: алгоритм Фалкерсона	21
2.3.2 FloydWarshall<T>: алгоритм Флойда-Уоршалла	22
2.3.3 Сравнение алгоритмов по числу итераций	23
2.4 Модуль lab3	23
2.4.1 MaxFlow<T>: алгоритм Форда–Фалкерсона (Эдмондс–Карп)	23
2.4.2 MinCostFlow<T>: поток минимальной стоимости	24
2.5 Модуль lab4	26
2.5.1 Kirchhoff: число остовных деревьев	26

2.5.2	Kruskal: алгоритм Краскала	27
2.5.3	Prufer: код Прюфера	27
2.5.4	MIS: максимальное независимое множество	28
2.6	Модуль lab5	29
2.6.1	EulerianCycle: проверка, достройка и построение цикла .	29
2.6.2	CycleBasis: фундаментальная система циклов	31
3	Результаты работы программы	33
3.1	Генерация графа (пункты 1–3)	33
3.2	Лабораторная работа 1 (пункты 4–8)	35
3.3	Лабораторная работа 2 (пункты 9–11)	36
3.4	Лабораторная работа 3 (пункты 12–15)	39
3.5	Лабораторная работа 4 (пункты 16–20)	42
3.6	Лабораторная работа 5 (пункты 21–23)	44
	Заключение	46
	Список литературы	48

Введение

Задание лабораторной работы 1.

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с заданным распределением (параметры распределения задаются как константы и подбираются экспериментально) с введенным пользователем количеством вершин. Распределение брать из справочника Вадзинского: реализовать генерирующую формулу распределения Вейбулла. Генерация происходит по степеням вершин.
2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для сгенерированной с помощью заданного распределения весовой матрицы (пользователь вводит количество рёбер), в ответе представить минимальный и/или максимальный путь (в виде матрицы). При генерации необходимо учесть генерацию только положительных значений, только отрицательных значений и смешанную (на выбор пользователя).
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указывать количество таковых маршрутов.
5. Реализовать меню для управления всеми пунктами.

Задание лабораторной работы 2.

1. Для заданных графов (случайно сгенерированных в предыдущей работе) реализовать упорядочивание графа по алгоритму Фалкерсона.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами – выбирает пользователь) и найти кратчайший путь для двух выбранных пользователем вершин. В результате выводится не только расстояние, но и сам путь в виде последовательности вершин. Реализовать алгоритм Флойда-Уоршалла с выводом матрицы расстояний.
3. Сравнить скорости работы реализованных алгоритмов (по количеству итераций).

Задание лабораторной работы 3.

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по алгоритму Форда–Фалкерсона.

3. Вычислить заданный поток минимальной стоимости (в качестве величины потока брать значение, равное $\lfloor 2/3 \cdot max \rfloor$, где max – максимальный поток). Для поиска кратчайшего пути по стоимости использовать алгоритм Флойда-Уоршалла.

Задание лабораторной работы 4.

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) найти число остовных деревьев, используя матричную теорему Кирхгофа.
2. Построить минимальный по весу остов для сгенерированного (неориентированного) взвешенного графа, используя алгоритм Краскала. Полученный остов закодировать с помощью кода Прюфера и декодировать его; веса сохраняются при кодировании.
3. Реализовать алгоритм нахождения максимального независимого множества вершин на исходном графе и полученном остове (по выбору пользователя).

Задание лабораторной работы 5.

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) проверить, является ли граф эйлеровым. Если нет, то модифицировать граф (логировать, что изменено). Построить эйлеров цикл.
2. Для заданных неориентированных графов (случайно сгенерированных в первой работе) построить кратчайший остов (алгоритмом из четвертой работы). На основе данного остова получить фундаментальную систему циклов (вывести ее на экран), получение всех остальных циклов на основе фундаментальных с помощью операции симметрической разности (выбранные циклы вводит пользователь).

1. Математическое описание

1.1. Граф и его определение

Графом $G(V, E)$ называется совокупность двух множеств – непустого множества V (множества *вершин*) и множества E двухэлементных подмножеств множества V (E – множество *рёбер*),

$$G(V, E) \stackrel{\text{Def}}{=} \langle V; E \rangle, \quad V \neq \emptyset, \quad E \subset 2^V \text{ \& } \forall e \in E (|e| = 2).$$

Если элементами множества E являются *упорядоченные* пары (т. е. $E \subset V \times V$), то граф называется *ориентированным* (или *орграфом*), в противном случае граф будет неориентированным. Ациклический граф не содержит циклов; связный ациклический неориентированный граф является деревом, ориентированный – DAG (directed acyclic graph).

1.2. Распределение классического Вейбулла

Для генерации случайных значений используется формула из учебника [3]

$$X = a (-\ln r)^{1/c}, \text{ где } r \in (0, 1).$$

В реализации используются два независимых экземпляра распределения с параметрами $a = 10$, $c = 2$: один для генерации структуры графа (степеней вершин), другой – для генерации весов рёбер.

На рисунке 1 показана график распределения из учебника.

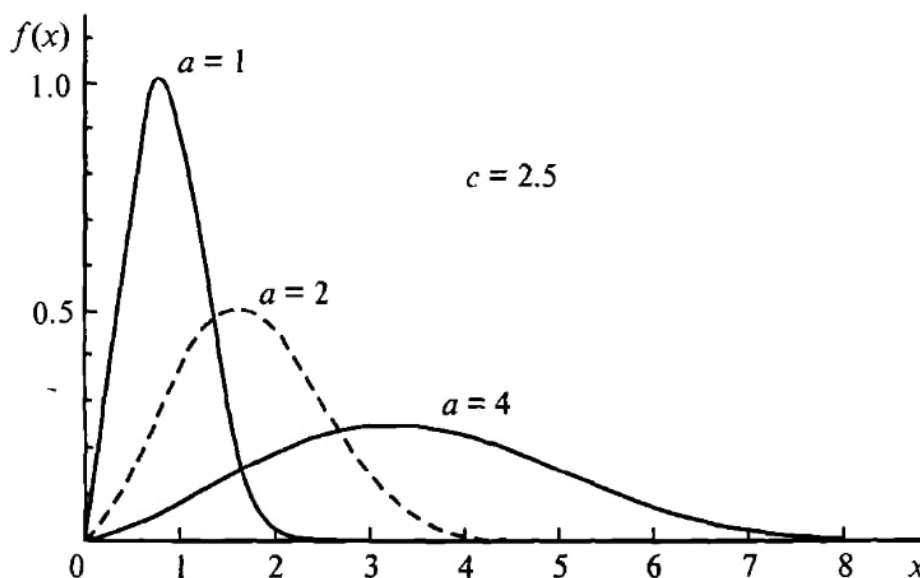


Рис. 1: Распределение Вейбулла

На рисунке 2 показана гистограмма 1000 сгенерированных значений и теоретическая плотность распределения.

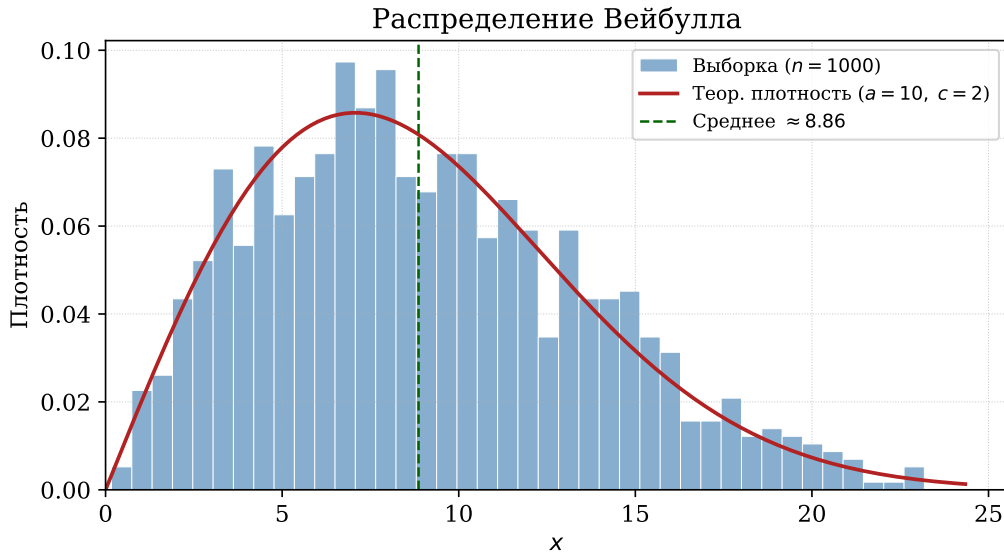


Рис. 2: Распределение Вейбулла ($a = 10$, $c = 2$): выборка $n = 1000$ и теоретическая плотность

1.3. Генерация связного ациклического графа по степеням вершин

Пользователь задаёт число вершин n , тип ориентированности и знак весов. Число рёбер не вводится напрямую.

Шаг 1: генерация целевых степеней. Для каждой вершины $i \in \{0, \dots, n - 1\}$ генерируется значение по распределению Вейбулла, округляемое до целого числа в диапазоне $[1, \lfloor \log n \rfloor]$:

$$d_i = \max(1, \min(\lfloor X_i \rfloor, \lfloor \log n \rfloor)), \quad X_i \sim \text{Weibull}(a, c).$$

Шаг 2: формирование остовного дерева. Для каждой i -й вершины при $i \geq 1$ случайный предок выбирается из числа вершин с номерами от 0 до $i - 1$, и добавляется направленное ребро от j -й вершины к i -й вершине. Поскольку $j < i$ для всех добавляемых рёбер, ни одно ребро не идёт «назад» в топологическом порядке, что гарантирует ациклическость. После этого шага граф связан и содержит ровно $n - 1$ рёбер.

Гарантия связности. Связность обеспечивается тем, что вершина 0 является корнем. На каждом шаге i новая вершина подключается ровно одним ребром к одной из уже добавленных вершин $\{0, \dots, i - 1\}$. Таким образом, после обработки всех n вершин каждая из них достижима из корня, и граф связан.

Шаг 3: добирание дополнительных рёбер. Для каждой i -й вершины формируется множество допустимых целей:

$$T_i = \{ j \mid j > i, (i, j) \notin E \}.$$

Множество T_i случайно перемешивается. Затем добавляются рёбра из перемешанного списка. Условие $j > i$ сохраняет инвариант ациклическости: все рёбра по-прежнему направлены в сторону возрастания индекса.

Учёт ориентированности и весов. Для неориентированного графа каждое ребро $u \rightarrow v$ добавляется симметрично: дополнительно добавляется $v \rightarrow u$ с тем же весом. Вес каждого ребра генерируется из и распределения Вейбулла и при необходимости меняет знак согласно выбранному режиму (POSITIVE, NEGATIVE, MIXED).

1.4. Эксцентриситет, центр и диаметральные вершины

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от вершины v до других вершин графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(v, u).$$

Наиболее эксцентричные вершины – это концы диаметра.

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v).$$

Вершина v называется *центральной*, если её эксцентриситет совпадает с радиусом графа, $e(v) = R(G)$.

Множество центральных вершин называется *центром* графа и обозначается $C(G)$:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}.$$

1.5. Метод Шимбелла

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме:

$$a \cdot b = b \cdot a \Rightarrow a + b = b + a, \quad a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \quad (1)$$

2. Операция сложения двух величин a и b заменяется выбором минимального (максимального) элемента из ненулевых:

$$a + b = b + a = \min(\max)\{a, b\}. \quad (2)$$

Нули при этом игнорируются: минимальный или максимальный элемент выбирается из ненулевых значений. Нуль в результате операции может быть получен лишь тогда, когда все слагаемые – нулевые.

Элемент матрицы $A^{(2)}$, полученной возведением A в степень 2, вычисляется как:

$$\omega_{ij}^{(2)} = \min(\max) \left\{ \omega_{i1}^{(1)} + \omega_{1j}^{(1)}, \omega_{i2}^{(1)} + \omega_{2j}^{(1)}, \dots, \omega_{in}^{(1)} + \omega_{nj}^{(1)} \right\}.$$

Матрица $A^{(k)}$, вычисленная последовательным умножением, содержит длины путей из ровно k рёбер для всех пар вершин.

1.6. Упорядочивание графа по алгоритму Фалкерсона

Алгоритм Фалкерсона – графический метод топологической сортировки DAG, состоящий из следующих шагов.

1. Найти вершины графа, в которые не входит ни одна дуга. Они образуют первую группу. Пронумеровать вершины группы в произвольном порядке.
2. Вычеркнуть все пронумерованные вершины и дуги, из них исходящие. В получившемся графе найдётся, по крайней мере, одна вершина, в которую не входит ни одна дуга. Этой вершине, входящей во вторую группу, присвоить очередной номер и так далее. Вторым шагом повторять до тех пор, пока не будут упорядочены все вершины.

1.7. Нахождение кратчайшего пути. Алгоритм Флойда-Уоршалла

Алгоритм Флойда-Уоршалла находит кратчайшие пути между *всеми* парами вершин в ориентированном взвешенном графе.

Для хранения информации о путях используется матрица H размера $n \times n$:

$$H[i, j] = \begin{cases} k, & \text{если } k \text{ – первая вершина, достигаемая на кратчайшем пути из } i \text{ в } j; \\ -1, & \text{если из вершины } i \text{ в вершину } j \text{ нет пути.} \end{cases}$$

Основной цикл: для каждой промежуточной вершины k проверяется

$$\text{если } d[i][k] + d[k][j] < d[i][j], \quad \text{то } d[i][j] \leftarrow d[i][k] + d[k][j], \quad H[i][j] \leftarrow H[i][k].$$

Если в G есть цикл с отрицательным весом, то решения поставленной задачи не существует.

1.8. Определение сети

Пусть $G = (V, E)$ — орграф с одним источником $s \in V$ и одним стоком $t \in V$, где $s \neq t$.

Сетью $S = (G; c)$ назовём орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное действительное число $c(e)$, называемое *пропускной способностью* этой дуги.

1.9. Поток в сети

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое *поток* по этой дуге e , для которой выполняются свойства:

1. для каждой дуги $e \in E$ верно $f(e) \leq c(e)$, т. е. поток по любой дуге не превышает пропускной способности этой дуги;
2. для каждой вершины $v \in V, v \neq s, t$, верно

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0,$$

т. е. поток, входящий в любую вершину, кроме источника и стока, без остатка выходит из неё (сохранение потока).

При этом неотрицательное число

$$p_f = \sum_{e \in A(s)} f(e)$$

называется **величиной потока** f .

1.10. Алгоритм Форда–Фалкерсона

Теорема (Л. Р. Форд, Д. Р. Фалкерсон, 1962). *Величина максимального потока $p_{\max}$ в сети S совпадает с минимальной пропускной способностью $r_{\min}$ её разрезом.*

Алгоритм ищет максимальный поток от истока до стока.

В начале остаточный граф инициализируется из матрицы весов – каждое ребро содержит свою пропускную способность, поток везде равен нулю.

Затем алгоритм итеративно ищет путь от истока до стока с помощью BFS – только по рёбрам у которых ещё есть свободное место. Если путь не найден – алгоритм завершается, максимальный поток найден.

Если путь найден, определяется узкое место – минимальная свободная пропускная способность среди всех рёбер пути. Именно столько можно дополнительно пустить по этому пути.

После этого остаточный граф обновляется: каждое прямое ребро пути уменьшается на величину узкого места – места стало меньше. Каждое обратное ребро увеличивается на ту же величину – это позволяет алгоритму на следующих итерациях пойти обратно по этому пути, если это будет выгодно. Найденная величина добавляется к общему потоку. Процесс повторяется до тех пор пока существует путь от истока до стока.

1.11. Нахождение заданного потока минимальной стоимости

Рассмотрим задачу определения потока заданной величины θ от s к t в сети $G = (S, U, \Omega, D)$, в которой каждая дуга (x_i, x_j) характеризуется пропускной способностью $c(x_i, x_j) \in \Omega$ и неотрицательной стоимостью $d(x_i, x_j) \in D$.

Математическая модель задачи: минимизировать

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j),$$

при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq c(x_i, x_j)$ для всех $(x_i, x_j) \in U$;
2. для истока s : $\sum_{x_j} \varphi(s, x_j) - \sum_{x_j} \varphi(x_j, s) = \theta$
3. для стока t : $\sum_{x_j} \varphi(t, x_j) - \sum_{x_j} \varphi(x_j, t) = -\theta$
4. для любой другой вершины: условие сохранения потока равно нулю.

1.12. Теорема Кирхгофа

Определим *матрицу Кирхгофа* $B = B(G) = (\beta_{ij})_{n \times n}$, полагая

$$B(G) = \begin{pmatrix} \deg v_1 & 0 & \dots & 0 \\ 0 & \deg v_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \deg v_n \end{pmatrix} - A(G),$$

где $A(G)$ — матрица смежности графа G . Иными словами,

$$\beta_{ij} = \begin{cases} -1, & \text{если } i \neq j \text{ и } v_i \text{ смежна с } v_j, \\ 0, & \text{если } i \neq j \text{ и } v_i \text{ не смежна с } v_j, \\ \deg v_i, & \text{если } i = j. \end{cases}$$

Теорема (матричная теорема Кирхгофа). Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$

1.13. Алгоритм Краскала

Алгоритм перебирает рёбра графа в порядке возрастания веса и жадно добавляет каждое ребро в остов, если оно не создаёт цикл.

Рёбра сортируются по весу, затем для каждого ребра (u, v) проверяется существование пути между u и v в уже построенной части дерева T . Если путь есть — добавление ребра создало бы цикл, поэтому оно пропускается. Если пути нет — ребро добавляется в T .

Алгоритм завершается когда $|T| = n - 1$, то есть все вершины связаны минимальным числом рёбер без циклов. Суммарный вес полученного остова:

$$W = \sum_{(u,v) \in T} w(u, v).$$

1.14. Код Прюфера

Кодирование (дерево $\rightarrow$ код Прюфера). Алгоритм итеративно удаляет листья дерева. На каждом шаге среди всех вершин находится лист с минимальным номером. Сосед этого листа записывается в последовательность кода, после чего лист удаляется из дерева и степени обновляются. Процесс повторяется $n - 1$ раза. В результате получается последовательность длины $n - 1$:

$$\text{код} = (a_1, a_2, \dots, a_{n-2}),$$

где каждый элемент a_i – сосед удалённого на шаге i листа.

Декодирование (код Прюфера $\rightarrow$ дерево). На каждом шаге среди всех активных вершин ищется минимальная, которая не встречается в оставшейся части кода – она является листом. Найденный лист соединяется ребром с текущим элементом кода, после чего лист помечается как удалённый и код сдвигается вперёд. После обработки всех $n - 1$ элементов кода две оставшиеся активные вершины соединяются последним ребром. В результате восстанавливается исходное дерево с $n - 1$ рёбрами.

1.15. Максимальное независимое множество вершин

Множество вершин $S \subset V$ графа $G(V, E)$ называется *доминирующим множеством* (или *внешне устойчивым*), если $S \cup \Gamma(S) = V$, то есть

$$\forall v \in V \left(v \in S \vee \exists s \in S ((s, v) \in E) \right).$$

Очевидно, что множество S доминирует тогда и только тогда, когда

$$\forall v \notin S \left(\Gamma(v) \cap S \neq \emptyset \right),$$

что равносильно утверждению $\forall v \in V \left(\exists s \in S (d(v, s) \leq 1) \right)$.

Множество вершин называется *независимым* (или *внутренне устойчивым*), если никакие две из них не смежны.

Теорема. Независимое множество вершин является максимальным тогда и только тогда, когда оно является доминирующим.

Алгоритм поиска максимального независимого множества. Алгоритм перебирает все возможные подмножества вершин с помощью рекурсивного возврата.

Начинаем с пустого множества S и полного списка кандидатов T , содержащего все вершины графа. Для каждой вершины v из списка кандидатов проверяем: есть ли ребро между v и хотя бы одной вершиной из S ? Если ребра нет – v можно добавить в S без нарушения независимости.

После добавления v в S формируется новый список кандидатов – из T убираются сама вершина v и все её соседи, так как они уже не могут войти в независимое множество вместе с v . Далее алгоритм рекурсивно продолжает перебор с обновлёнными S и T .

Если размер текущего S превышает размер лучшего найденного множества – сохраняем S как новый ответ.

Когда список кандидатов T опустеет – ветка заканчивается и алгоритм возвращается назад, пробуя следующую вершину. После перебора всех ветвей в S^* будет лежать максимальное независимое множество.

1.16. Эйлеровы графы

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется *эйлеровым циклом*, а граф называется *эйлеровым графом*.

Если граф имеет цепь (не обязательно простую), содержащую все рёбра, то такая цепь называется *эйлеровой цепью*, а граф называется *полуэйлеровым графом*.

Эйлеров цикл содержит не только все рёбра (по одному разу), но и все вершины графа (возможно, по несколько раз).

Эйлеровым может быть только связный граф.

Теорема: Если граф G связан и нетривиален, то следующие утверждения эквивалентны:

1. G - эйлеров граф.
2. Каждая вершина G имеет чётную степень.
3. Множество рёбер G можно разбить на простые циклы.

1.17. Достройка до эйлерового графа

Алгоритм итеративно устраняет вершины нечётной степени.

На каждой итерации собирается список вершин с нечётной степенью. Если список пуст – граф уже эйлеров, алгоритм завершается.

Иначе перебираются все пары нечётных вершин (u, v) и для каждой пары выполняется одно из трёх действий:

1. если ребро (u, v) отсутствует – добавляем его. Степени обеих вершин увеличиваются на 1 и становятся чётными;

2. если ребро (u, v) существует и не является мостом – удаляем его. Степени обеих вершин уменьшаются на 1 и становятся чётными. Мост проверяется через BFS: ребро (u, v) удаляется из копии матрицы смежности, после чего проверяется достижимость v из u ; если v недостижима – ребро является мостом;
3. если ребро (u, v) существует и является мостом – пара пропускается, переходим к следующей.

Если за одну итерацию не удалось исправить ни одну пару – алгоритм завершается. По завершении граф является эйлеровым (все степени чётные), полуэйлеровым (ровно две нечётные вершины) или не эйлеровым (более двух нечётных вершин).

1.18. Алгоритм построения эйлерова цикла

Алгоритм построения эйлерова цикла. Алгоритм строит эйлеров цикл с помощью стека.

В качестве стартовой выбирается любая вершина с ненулевой степенью. Стартовая вершина помещается в стек S .

Пока стек не пуст выполняется следующее. Смотрим на верхнюю вершину v стека. Если из v есть хотя бы одно непройденное ребро – берём первого доступного соседа u , помещаем u в стек и удаляем ребро (v, u) из копии матрицы смежности. Если из v нет непройденных рёбер – извлекаем v из стека и добавляем в результат.

После завершения проверяется корректность: если длина результата не равна числу рёбер плюс один – цикл не найден. В противном случае результат содержит последовательность вершин эйлерова цикла.

1.19. Фундаментальная система циклов и циклический ранг

Пусть $T(V, E_T)$ – некоторый остов графа $G(V, E)$. Кодеревом $T^*(V, E_T^*)$ остова T называется остовный подграф, такой что $E_T^* = E \setminus E_T$. Рёбра кодерева называются *хордами* остова. Каждая хорда $e \in E_T^*$ остова T порождает ровно один простой цикл, обозначим его Z_e . Таким образом, имеем систему простых циклов

$$Z \stackrel{\text{Def}}{=} \{Z_e\}_{e \in E_T^*},$$

определяемых выбранным остовом T , которая называется *фундаментальной системой циклов*. Циклы фундаментальной системы называются *фундаментальными циклами*, а количество циклов в данной фундаментальной системе называется *циклическим рангом* (или *цикломатическим числом*) графа G и обозначается $m(G)$.

1.20. Симметрическая разность циклов

Любой цикл в связном графе $G(V, E)$ можно представить как симметрическую разность нескольких фундаментальных циклов из системы Z , определяемой произвольным остовом T .

Симметрической разностью двух множеств рёбер Z_a и Z_b называется

$$Z_a \triangle Z_b = (Z_a \setminus Z_b) \cup (Z_b \setminus Z_a),$$

то есть множество рёбер, принадлежащих ровно одному из двух циклов.

2. Особенности реализации

Проект собирается через CMake. Точка входа – единый исполняемый файл с консольным меню. Все модули лабораторных зависят только от `common`; `common` ничего не знает про лабораторные.

Пользователь выбирает пункт меню, `main.cpp` вызывает нужный метод `LabXUI`, который читает параметры, запускает алгоритм и выводит результат в консоль.

2.1. Модуль `common`

Модуль содержит реализацию графа, генератор случайных чисел по распределению Вейбулла и генератор графов. Он используется всеми лабораторными для переиспользования кода.

2.1.1. `Graph<T>`: хранение графа

Вход: число вершин n ; рёбра добавляются через `addEdge(from, to, weight)`.

Выход: матрицы смежности и весов, методы обхода (BFS, DFS).

Описание: `Graph<T>` – шаблонный класс; тип весов рёбер задаётся параметром T . Граф хранится в двух матрицах размера $n \times n$: `adjMatrix[i][j] = 1` означает наличие ребра (i, j) , `weightMatrix[i][j]` хранит его вес.

```
1 template <typename T>
2 class Graph {
3     private:
4         int numVertices;
5         vector<vector<int>> adjMatrix;
6         vector<vector<T>> weightMatrix;
7     public:
8         Graph(int n);
9         void addEdge(int from, int to, T weight);
10        bool hasEdge(int from, int to) const;
11        T getEdge(int from, int to) const;
12        int getNumVertices() const;
13        vector<vector<int>> getAdjMatrix() const;
14        vector<vector<T>> getWeightMatrix() const;
15        Graph<T> reorder(const vector<int>& order) const;
16        // ...
17};
```

Listing 1: Объявление `Graph` (`graph.h`)

Также реализованы BFS и DFS внутри класса, методы подсчёта эксцентриситетов и метод `reorder(order)`, который перестраивает граф по новому порядку вершин – используется для вывода матрицы после алгоритма Фалкерсона.

Сложность: проверка ребра и доступ к весу – $O(1)$; BFS по матрице – $O(V^2)$; память – $O(V^2)$.

2.1.2. WeibullDistribution: генерация случайных чисел

Вход: параметры масштаба a и формы c (по умолчанию $a = 10, c = 0.2$).

Выход: случайное число $X > 0$ по распределению Вейбулла.

Описание: Класс хранит параметры и равномерный генератор `std::mt19937`. Метод `generate()` берёт $u \in (0, 1)$ и считает $X = a \cdot (-\ln u)^{1/c}$. Модуль нужен, чтобы результат был положительным.

```
1 double WeibullDistribution::generate() {  
2     double u = distribution(generator);  
3     double raw = a * std::pow(-std::log(u), 1.0 / c);  
4     return (raw > 0) ? raw : -raw;  
5 }
```

Listing 2: Генерация числа по распределению Вейбулла (weibull.cpp)

Сложность: $O(1)$ на одну генерацию.

2.1.3. Generator<T>: генерация графов

Вход: число вершин, тип ориентированности, тип знака весов, параметры двух распределений Вейбулла (для структуры и для весов).

Выход: `Graph<T>` – связный ациклический граф.

Описание: `Generator<T>` держит два объекта `WeibullDistribution`: для структуры графа и для весов рёбер. Метод `generateGraph()` строит граф в два прохода.

В первом проходе строится остовное дерево: вершина i соединяется с одной из вершин $\{0, \dots, i - 1\}$ – это гарантирует связность и ациклическость. Во втором проходе для каждой вершины с оставшимся бюджетом степени берутся все вершины $j > i$, у которых ещё нет ребра от i . Список перемешивается через `weibullShuffle` и добавляются первые d_i рёбер.

```
1 template <typename T>  
2 Graph<T> Generator<T>::generateGraph() {  
3     Graph<T> graph(numVertices);  
4  
5     int maxDeg = max(1, static_cast<int>(log(numVertices)));  
6     vector<int> degrees(numVertices);  
7     for (int i = 0; i < numVertices; i++) {  
8         int raw =  
9         static_cast<int>(distributionForStructure.generate());  
10        degrees[i] = max(1, min(raw, maxDeg));  
11    }  
12  
13    vector<int> perm(numVertices);  
14    iota(perm.begin(), perm.end(), 0);  
15    weibullShuffle(perm);
```

```

15
16 for (int i = 1; i < numVertices; i++) {
17     int j = randomIndex(i);
18     T weight = generateWeight(weightMode);
19     graph.addEdge(perm[j], perm[i], weight);
20     if (!directed) {
21         graph.addEdge(perm[i], perm[j], weight);
22     }
23 }
24
25
26 for (int i = 0; i < numVertices; i++) {
27     int edgesToAdd = degrees[i];
28     vector<int> possibleTargets;
29
30     for (int j = i + 1; j < numVertices; j++) {
31         if (!graph.hasEdge(perm[i], perm[j])) {
32             possibleTargets.push_back(j);
33         }
34     }
35
36     if (!possibleTargets.empty()) {
37         weibullShuffle(possibleTargets);
38
39         for (int k = 0; k < edgesToAdd && k <
40             (int)possibleTargets.size(); k++) {
41             int target = possibleTargets[k];
42             T weight = generateWeight(weightMode);
43             graph.addEdge(perm[i], perm[target], weight);
44             if (!directed) {
45                 graph.addEdge(perm[target], perm[i], weight);
46             }
47         }
48     }
49
50     return graph;
51 }

```

Listing 3: Генерация графа (generator.cpp)

Сложность: $O(V^2)$ по времени; $O(V^2)$ по памяти.

2.2. Модуль lab1

2.2.1. Эксцентриситеты, центр и диаметральные вершины

Вход: Graph<T> – связный граф.

Выход: вектор эксцентриситетов всех вершин; список центральных вершин (минимальный эксцентриситет); список диаметральных вершин (максимальный эксцентриситет).

Описание: Для каждой вершины запускается BFS по матрице весов – ребро считается существующим, если вес ненулевой. BFS возвращает расстояния в рёбрах. Эксцентриситет – наибольшее расстояние до других вершин. Центр – вершины с минимальным эксцентриситетом, диаметральные – с максимальным.

```
1 template <typename T>
2 int Graph<T>::eccentricity(int vertex) const {
3     vector<int> dist = bfs(vertex);
4     const int INF = std::numeric_limits<int>::max();
5     int maxDist = 0;
6     for (int i = 0; i < numVertices; i++) {
7         if (i == vertex || dist[i] == INF) continue;
8         maxDist = std::max(maxDist, dist[i]);
9     }
10    return maxDist;
11 }
```

Listing 4: BFS и вычисление эксцентриситета (graph.cpp)

Сложность: $O(V^2)$ по времени, $O(V)$ по памяти.

```
1 template <typename T>
2 vector<int> Graph<T>::allEccentricities() const {
3     vector<int> eccs(numVertices);
4     for (int i = 0; i < numVertices; i++) {
5         eccs[i] = eccentricity(i);
6     }
7     return eccs;
8 }
9 template <typename T>
10 vector<int> Graph<T>::findCenter() const {
11     vector<int> eccs = allEccentricities();
12     int radius = *std::min_element(eccs.begin(), eccs.end());
13     vector<int> centers;
14     for (int i = 0; i < numVertices; i++)
15         if (eccs[i] == radius) centers.push_back(i);
16     return centers;
17 }
18
19 template <typename T>
20 vector<int> Graph<T>::findDiametral() const {
21     vector<int> eccs = allEccentricities();
22     int diameter = 0;
23     for (int ecc : eccs)
24         if (ecc != std::numeric_limits<int>::max() && ecc >
25             diameter)
26             diameter = ecc;
27     vector<int> diametral;
28     for (int i = 0; i < numVertices; i++)
29         if (eccs[i] == diameter) diametral.push_back(i);
30     return diametral;
31 }
```

Listing 5: Поиск центра и диаметральных вершин (graph.cpp)

Сложность: $O(V^3)$ по времени, $O(V)$ по памяти.

2.2.2. Shimbelsolver<T>: метод Шимбелла

Вход: Graph<T> const&, int k – длина пути в рёбрах.

Выход: матрица минимальных и матрица максимальных путей ровно из k рёбер для всех пар вершин.

Описание: Начальная матрица заполняется по весам графа: если ребро (i, j) есть – ставится $w(i, j)$, иначе – INF. Матрица умножается $k - 1$ раз: для каждой пары (i, j) перебираются промежуточные вершины m и выбирается минимальный или максимальный кандидат $\text{mat}[i][m] + w[m][j]$.

```
1 template <typename T>
2 void Shimbelsolver<T>::multiplyMatrix(bool findMin) {
3     const T INF = getINF();
4     const T empty = findMin ? INF : -INF;
5     auto weight = graph.getWeightMatrix();
6     vector<vector<T>> next(n, vector<T>(n, empty));
7     for (int i = 0; i < n; i++)
8         for (int j = 0; j < n; j++)
9             for (int m = 0; m < n; m++) {
10                 if (currentMatrix[i][m] == empty
11                     || weight[m][j] == T{}) continue;
12                 T candidate = currentMatrix[i][m] + weight[m][j];
13                 if (findMin) next[i][j] = std::min(next[i][j],
14 candidate);
15                 else next[i][j] = std::max(next[i][j],
16 candidate);
17             }
18     currentMatrix = next;
19 }
```

Listing 6: Матричное умножение для метода Шимбелла (shimbel.cpp)

Сложность: $O(k \cdot V^3)$ по времени; $O(V^2)$ по памяти.

2.2.3. Поиск всех маршрутов (DFS с возвратом)

Вход: Graph<T> const&, вершина from, вершина to.

Выход: vector<vector<int>> – список всех простых путей из from в to.

Описание: findAllPaths запускает рекурсивный DFS с возвратом. При достижении целевой вершины текущий путь сохраняется в результат.

```
1 template <typename T>
2 vector<vector<int>> Graph<T>::findAllPaths(int from, int to) const
3 {
4     vector<vector<int>> allPaths;
5 }
```

```

4  vector<bool> visited(numVertices, false);
5  vector<int> currentPath = {from}; // путь начинается с from
6
7  visited[from] = true;
8  dfs(from, to, visited, currentPath, allPaths);
9
10 return allPaths;
11 }

```

Listing 7: Поиск всех маршрутов (graph.cpp)

Сложность: по времени $O(V!)$, по памяти $O(V)$.

2.3. Модуль lab2

2.3.1. Fulkerson<T>: алгоритм Фалкерсона

Вход: Graph<T> const& – связный ориентированный ациклический граф.

Выход: vector<int> – новые номера вершин после топологической сортировки.

Описание: На каждой итерации метод findRoots просматривает матрицу смежности и находит все ещё не удалённые вершины без входящих дуг. Каждой такой вершине присваивается очередной номер, затем она помечается удалённой. Шаги повторяются, пока не упорядочены все вершины. Счётчик iterCount растёт при каждом просмотре ячейки матрицы. Если вершин без входящих дуг нет, а вершины ещё остались – в графе есть цикл и сортировка невозможна.

```

1  template <typename T>
2  vector<int> Fulkerson<T>::findRoots(
3      const vector<bool>& removed) const
4  {
5      auto adj = graph.getAdjMatrix();
6      vector<int> roots;
7      for (int v = 0; v < n; v++) {
8          if (removed[v]) continue;
9          bool hasIncoming = false;
10         for (int from = 0; from < n; from++) {
11             iterCount++;
12             if (!removed[from] && from != v
13                 && adj[from][v] != 0) {
14                 hasIncoming = true;
15                 break;
16             }
17         }
18         if (!hasIncoming) roots.push_back(v);
19     }
20     return roots;
21 }
22
23 template <typename T>

```

```

24 vector<int> Fulkerson<T>::computeOrder() const {
25     iterCount = 0;
26     vector<bool> removed(n, false);
27     vector<int> order(n, -1);
28     int counter = 0;
29     while (counter < n) {
30         vector<int> roots = findRoots(removed);
31         if (roots.empty()) { /* цикл в графе */ return {}; }
32         for (int v : roots) {
33             order[v] = counter++;
34             removed[v] = true;
35         }
36     }
37     return order;
38 }

```

Listing 8: Поиск корней и вычисление порядка (fulkerson.cpp)

После сортировки вызывается `Graph<T>::reorder(order)`, и выводятся перенумерованные матрицы смежности и весов.

Сложность: $O(V^2)$ по времени; $O(V)$ по памяти.

2.3.2. FloydWarshall<T>: алгоритм Флойда-Уоршалла

Вход: `Graph<T> const&` – ориентированный взвешенный граф.

Выход: матрица расстояний `distMatrix` и матрица следующих вершин `nextMatrix` для восстановления пути.

Описание: Инициализация: $d[i][i] = 0$, $d[i][j] = w(i, j)$ если ребро есть, иначе $+\infty$; $next[i][j] = j$ при наличии ребра, -1 – иначе. Главный тройной цикл перебирает промежуточные вершины k и обновляет расстояния.

```

1  template <typename T>
2  void FloydWarshall<T>::compute() {
3      const T INF = getPosINF();
4      iterCount = 0;
5      for (int k = 0; k < n; k++)
6          for (int i = 0; i < n; i++)
7              for (int j = 0; j < n; j++) {
8                  iterCount++;
9                  if (distMatrix[i][k] == INF
10                     || distMatrix[k][j] == INF) continue;
11                  T newDist = distMatrix[i][k] + distMatrix[k][j];
12                  if (newDist < distMatrix[i][j]) {
13                      distMatrix[i][j] = newDist;
14                      nextMatrix[i][j] = nextMatrix[i][k];
15                  }
16              }
17  }

```

Listing 9: Основной цикл Флойда-Уоршалла (warshall.cpp)

Путь восстанавливается методом `getPath(from, to)`: идём по `nextMatrix` от `from` до `to`.

Сложность: $O(V^3)$ по времени; $O(V^2)$ по памяти.

2.3.3. Сравнение алгоритмов по числу итераций

Вход: `Graph<T> const&`, на котором запускаются оба алгоритма.

Выход: счётчики `iterCount` Фалкерсона и Флойда-Уоршалла, выведенные на экран.

Описание: Пункт меню 11 запускает оба алгоритма на одном графе и сравнивает счётчики – каждый из них растёт при каждом просмотре ячейки матрицы.

2.4. Модуль lab3

2.4.1. `MaxFlow<T>`: алгоритм Форда–Фалкерсона (Эдмондс–Карп)

Вход: `Graph<T> const&`, исток `int s`, сток `int t`.

Выход: `T` – значение максимального потока.

Описание: Конструктор строит остаточный граф `cap` из весовой матрицы; отрицательные веса берутся по модулю, потому что пропускные способности должны быть неотрицательными.

Метод `bfs(s, t)` ищет кратчайший увеличивающий путь в остаточной сети – ребро (v, u) берётся только при `cap[v][u] > 0`. Главный цикл `compute(s, t)`:

1. ищет увеличивающий путь через BFS;
2. находит узкое место – минимальную пропускную способность на пути;
3. обновляет остаточный граф: прямые рёбра уменьшает, обратные увеличивает;
4. прибавляет найденный поток к итогу.

```
1 template <typename T>
2 T MaxFlow<T>::compute(int s, int t) {
3     T maxFlow = T{};
4     while (true) {
5         vector<int> path = bfs(s, t);
6         if (path.empty()) break;
7
8         T push = numeric_limits<T>::max();
9         for (int i = 0; i + 1 < (int)path.size(); i++)
10             push = min(push, cap[path[i]][path[i+1]]);
11
12         for (int i = 0; i + 1 < (int)path.size(); i++) {
13             cap[path[i]][path[i+1]] -= push;
```

```

14         cap[path[i+1]][path[i]] += push;
15     }
16     maxFlow += push;
17 }
18 return maxFlow;
19 }

```

Listing 10: Алгоритм Эдмондса–Карпа (maxflow.cpp)

Сложность: $O(V^3)$ по времени; $O(V^2)$ по памяти.

2.4.2. MinCostFlow<T>: поток минимальной стоимости

Вход: Graph<T>, матрица стоимостей vector<vector<T>>, исток s , сток t , целевой поток targetFlow.

Выход: пара (достигнутый поток, минимальная стоимость).

Описание: Конструктор заполняет остаточные пропускные способности cap и матрицу стоимостей cost; обратные рёбра получают противоположный знак стоимости.

На каждой итерации метод findMinCostPath(s , t):

1. строит вспомогательный граф residual из рёбер с ненулевой остаточной пропускной способностью;
2. запускает FloydWarshall<T> для поиска пути минимальной стоимости от s до t ;
3. возвращает путь и его стоимость.

```

1  template <typename T>
2  pair<vector<int>, T> MinCostFlow<T>::findMinCostPath(
3      int s, int t) const
4  {
5      Graph<T> residual(n);
6      for (int i = 0; i < n; i++)
7          for (int j = 0; j < n; j++)
8              if (cap[i][j] > T{})
9                  residual.addEdge(i, j, cost[i][j]);
10
11     FloydWarshall<T> fw(residual);
12     fw.compute();
13     const auto& dist = fw.getDistMatrix();
14     const T INF_VAL = numeric_limits<T>::max() / 2;
15     if (dist[s][t] >= INF_VAL) return {{}, T{}};
16     return {fw.getPath(s, t), dist[s][t]};
17 }
18
19 template <typename T>
20 pair<T, T> MinCostFlow<T>::compute(int s, int t, T targetFlow) {
21     T flow = T{};
22     T totalCost = T{};

```



```

23  int iter = 1;
24
25  while (flow < targetFlow) {
26      printIterationHeader(iter++, flow, targetFlow);
27      printMatrix(cap, "Остаточные пропускные способности:");
28
29      const T INF_VAL = numeric_limits<T>::max() / 2;
30      vector<vector<T>> resCost(n, vector<T>(n, INF_VAL));
31      for (int i = 0; i < n; i++)
32          for (int j = 0; j < n; j++)
33              if (cap[i][j] > T{})
34                  resCost[i][j] = cost[i][j];
35      printMatrix(resCost, "Стоимости рёбер остаточной сети:");
36
37      auto [path, pathCost] = findMinCostPath(s, t);
38
39      if (path.empty()) {
40          cout << "\n Путь из " << s << " в " << t
41              << " не найден - алгоритм завершён.\n";
42          break;
43      }
44
45      cout << "\n Кратчайший путь: ";
46      for (int i = 0; i < (int)path.size(); i++) {
47          if (i > 0)
48              cout << " => ";
49          cout << path[i];
50      }
51      cout << "    стоимость ( единицы потока: " << pathCost << ")\n";
52
53      T push = targetFlow - flow;
54      int bottleneckU = path[0], bottleneckV = path[1];
55      for (int i = 0; i + 1 < (int)path.size(); i++) {
56          if (cap[path[i]][path[i + 1]] <
57              push) {
58              push = cap[path[i]][path[i + 1]];
59              bottleneckU = path[i];
60              bottleneckV = path[i + 1];
61          }
62      }
63      cout << "    Узкое место: ребро " << bottleneckU << " => " <<
bottleneckV
64          << "    пропускная ( способность: " << push << ")\n";
65
66      for (int i = 0; i + 1 < (int)path.size(); i++) {
67          int u = path[i], v = path[i + 1];
68          cap[u][v] -= push;
69          cap[v][u] += push;
70      }
71
72      flow += push;
73      totalCost += push * pathCost;
74

```

```

75     cout << "    Пушено: " << push << "        Итого поток: " << flow
76         << "        Итого стоимость: " << totalCost << "\n";
77 }
78
79 return {flow, totalCost};
80 }

```

Listing 11: Поиск пути минимальной стоимости (mincost.cpp)

По найденному пути определяется узкое место, поток направляется по пути, остаточный граф обновляется. Итерации продолжаются до достижения целевого потока $\lfloor 2/3 \cdot F_{\max} \rfloor$ или пока путь есть.

Сложность: $O(F \cdot V^3)$ по времени, где F – целевой поток (каждая итерация запускает Флойда-Уоршалла); $O(V^2)$ по памяти.

2.5. Модуль lab4

2.5.1. Kirchhoff: число остовных деревьев

Вход: `Graph<int> const&` – неориентированный граф.

Выход: `long long` – число различных остовных деревьев.

Описание: Метод `count` строит матрицу Лапласа B размера $n \times n$: $B[i][j] = -1$ для смежных вершин $i \neq j$, $B[i][i]$ – степень вершины i . Смежность определяется как $\text{adj}[i][j] \neq 0$ или $\text{adj}[j][i] \neq 0$.

Затем берётся алгебраическое дополнение B' – матрица B без нулевой строки и нулевого столбца размера $(n - 1) \times (n - 1)$.

```

1 for (int k = 0; k < m; k++) {
2     int pivot = -1;
3     for (int i = k; i < m; i++)
4         if (mat[i][k] != 0) { pivot = i; break; }
5     if (pivot == -1) return 0;
6     if (pivot != k) swap(mat[k], mat[pivot]);
7
8     for (int i = k + 1; i < m; i++) {
9         for (int j = k + 1; j < m; j++) {
10             mat[i][j] = mat[k][k] * mat[i][j]
11                 - mat[i][k] * mat[k][j];
12             if (k > 0)
13                 mat[i][j] /= mat[k-1][k-1];
14         }
15         mat[i][k] = 0;
16     }
17 }
18 return abs(mat[m-1][m-1]);

```

Listing 12: Алгоритм Барейсса(kirchhoff.cpp)

Сложность: $O(V^3)$ по времени; $O(V^2)$ по памяти.

2.5.2. Kruskal: алгоритм Краскала

Вход: `Graph<int> const&` – связный неориентированный взвешенный граф.

Выход: `vector<Edge>` – рёбра минимального остова.

Описание: Конструктор собирает все рёбра (каждое один раз, условие $i < j$) в вектор `edges`; веса берутся по модулю. Метод `compute()` сортирует рёбра по весу и добавляет их в остов, если они не создают цикла. Цикл проверяется через BFS по уже построенному остову: если вершины u и v уже связаны – ребро пропускается.

```
1 vector<Edge> Kruskal::compute() {
2     sort(edges.begin(), edges.end());
3     vector<Edge> T;
4     Graph<int> tree(n);
5     for (auto& e : edges) {
6         if ((int)T.size() == n - 1) break;
7         if (hasPath(e.u, e.v, tree)) continue;
8         T.push_back(e);
9         tree.addEdge(e.u, e.v, e.w);
10        tree.addEdge(e.v, e.u, e.w);
11    }
12    return ((int)T.size() == n - 1) ? T : vector<Edge>{};
13 }
```

Listing 13: Основной цикл алгоритма Краскала (kruskal.cpp)

Сложность: $O(V^4)$ по времени; $O(V)$ по памяти.

2.5.3. Prufer: код Прюфера

Вход (encode): `vector<Edge> const&` – рёбра МОД, `int n` – число вершин.

Выход (encode): `PruferCode` – поля: `seq` (последовательность длины $n - 2$), `weights` (веса рёбер), `n`.

Описание (encode): Строится временный граф из рёбер МОД. Хранится вектор степеней `d` и вектор активных вершин `V`. На каждом шаге ищется активная висющаяся вершина с минимальным номером ($d[v] = 1$); её единственный сосед записывается в последовательность, вес ребра – в `weights`. Вершина v удаляется.

```
1 PruferCode Prufer::encode(const vector<Edge>& mst, int p) {
2     for (int i = 0; i < p - 1; i++) {
3         int v = -1;
4         for (int k = 0; k < p; k++)
5             if (V[k] && d[k] == 1) { v = k; break; }
6         int neighbor = -1, weight = 0;
7         for (int to = 0; to < p; to++)
8             if (V[to] && adj[v][to] != 0)
9                 { neighbor = to; weight = wgt[v][to]; break; }
10        code.seq.push_back(neighbor);
```

```

11     code.weights.push_back(weight);
12     V[v] = false; d[v]--; d[neighbor]--;
13 }
14 return code;
15 }

```

Listing 14: Кодирование в код Прюфера с сохранением весов (prufer.cpp)

Сложность: $O(V^2)$ по времени; $O(V^2)$ по памяти.

Вход (decode): PruferCode – структура с полями seq, weights, n.

Выход (decode): vector<Edge> – рёбра восстановленного дерева с весами.

Описание (decode): Поддерживается вектор активных вершин В. Для каждого элемента seq[i] ищется наименьшая неудаленная вершина, которой нет в остатке кода seq[i..n-2]. Между ней и seq[i] добавляется ребро с весом weights[i], вершина удаляется.

```

1 vector<Edge> Prufer::decode(const PruferCode& code) {
2     int p = code.n;
3     vector<Edge> E;
4     vector<bool> B(p, true);
5
6     for (int i = 0; i < p - 1; i++) {
7         int v = -1;
8         for (int k = 0; k < p; k++) {
9             if (B[k]) {
10                 bool in_rest = false;
11                 for (int j = i; j < p - 1; j++)
12                     if (code.seq[j] == k) { in_rest = true; break; }
13
14                 if (!in_rest) { v = k; break; }
15             }
16             E.push_back({v, code.seq[i], code.weights[i]});
17             B[v] = false;
18         }
19         return E;
20 }

```

Listing 15: Декодирование кода Прюфера (prufer.cpp)

Сложность: $O(V^2)$ по времени; $O(V)$ по памяти.

2.5.4. MIS: максимальное независимое множество

Вход: vector<vector<int>> const& – матрица смежности исходного графа или МОД (по выбору пользователя).

Выход: vector<int> – вершины максимального независимого множества.

Описание: Метод `compute()` запускает рекурсивный `backtrack(S, T)`, где S – текущее независимое множество, T – кандидаты. Для каждой вершины $v \in T$ проверяется, не смежна ли она с вершинами из S . Если нет – добавляем v и убираем из кандидатов всех соседей v . Если новое множество больше текущего максимума – сохраняем.

```

1
2 bool MIS::isIndependent(const vector<int> &S, int v) const {
3     for (int u : S) {
4         if (adj[u][v] != 0 || adj[v][u] != 0) {
5             return false; // множество S + v зависимое
6         }
7     }
8     return true;
9 }
10 void MIS::backtrack(vector<int> S, vector<int> T) {
11     for (int v : T) {
12         if (isIndependent(S, v)) {
13             vector<int> S_plus_v = S;
14             S_plus_v.push_back(v);
15             if ((int)S_plus_v.size() > m) {
16                 X = S_plus_v;
17                 m = S_plus_v.size();
18             }
19             vector<int> newT;
20             for (int u : T)
21                 if (u != v && adj[v][u] == 0 && adj[u][v] == 0)
22                     newT.push_back(u);
23             backtrack(S_plus_v, newT);
24         }
25     }
26 }

```

Listing 16: Исходный код (mis.cpp)

Сложность: $O(2^V)$ по времени в худшем случае; $O(V)$ по памяти.

2.6. Модуль lab5

2.6.1. EulerianCycle: проверка, достройка и построение цикла

Вход: `Graph<int> const&` – неориентированный граф.

Выход: модифицированный граф после `makeEulerian`; `optional<vector<int>` последовательность вершин эйлера цикла (или путь, если граф полуэйлеров).

Описание: Вспомогательные методы: `degree(v)` суммирует строку матрицы смежности; `oddVertices()` возвращает список вершин с нечётной степенью; `isConnected()` делает BFS и проверяет достижимость всех ненулевых вершин; `isBridge(u, v)` временно убирает ребро и проверяет через BFS, осталась ли связность.

Граф является эйлеровым, если он связан и все степени чётны:

```

1 bool EulerianCycle::isEulerian() const {
2     return isConnected() && oddVertices().empty();
3 }

```

Listing 17: Проверка эйлеровости (euler.cpp)

Метод `makeEulerian()` итеративно устраняет нечётные степени. Для каждой пары нечётных вершин (u, v) выполняется одно из трёх действий: добавить ребро, если его нет; удалить ребро, если оно не мост; пропустить пару, если ребро является мостом.

```

1 for (size_t i = 0; i < odd.size() && !fixed; i++) {
2     for (size_t j = i + 1; j < odd.size() && !fixed; j++) {
3         int u = odd[i], v = odd[j];
4         if (!g.hasEdge(u, v)) {
5             g.addEdge(u, v, 1); g.addEdge(v, u, 1);
6             addedEdges.emplace_back(u, v);
7             fixed = true;
8         } else if (!isBridge(u, v)) {
9             g.removeEdge(u, v); g.removeEdge(v, u);
10            fixed = true;
11        }
12    }
13 }

```

Listing 18: Достройка до эйлерового графа (euler.cpp)

Сложность: `makeEulerian` На каждой итерации перебираются все пары нечётных вершин — $O(V^2)$, для каждой пары вызывается `isBridge` через BFS — $O(V^2)$. Итераций не более $O(V)$, итого $O(V^5)$ по времени, $O(V^2)$ по памяти.

Метод `buildCycle()` строит эйлеров цикл алгоритмом Хирхольцера на основе стека. Пройденные рёбра обнуляются в рабочей копии матрицы.

```

1 S.push(start);
2 while (!S.empty()) {
3     int v = S.top(), u = -1;
4     for (int j = 0; j < n; j++)
5         if (adj[v][j]) { u = j; break; }
6     if (u == -1) { S.pop(); circuit.push_back(v); }
7     else { S.push(u); adj[v][u] = adj[u][v] = 0; }
8 }
9 reverse(circuit.begin(), circuit.end());

```

Listing 19: Построение эйлерова цикла (euler.cpp)

Сложность: `buildCycle` Каждое ребро обрабатывается ровно один раз, однако поиск следующего ребра требует просмотра строки матрицы — $O(V)$ на каждый шаг. Итого $O(V \cdot E) = O(V^3)$ по времени, $O(V^2)$ по памяти.

2.6.2. CycleBasis: фундаментальная система циклов

Вход: `vector<Edge> const&` – рёбра МОД (из Краскала), `Graph<int> const&` – исходный граф.

Выход: `vector<EdgeSet>` – система фундаментальных циклов; `EdgeSet` – результат симметрической разности выбранных циклов.

Описание: Метод `compute()` перебирает все рёбра исходного графа. Для каждого ребра (i, j) , не входящего в МОД (хорды), вызывается `pathInMST(i, j)` – BFS по остову, возвращающий путь. Из этого пути формируется множество рёбер; к нему добавляется хорда – получается фундаментальный цикл Z_e .

```
1 void CycleBasis::compute(const vector<Edge> &mst, const Graph<int>
   &g) {
2     n = g.getNumVertices();
3     basis.clear();
4
5     vector<vector<bool>> inTree(n, vector<bool>(n, false));
6     for (auto &e : mst) {
7         inTree[e.u][e.v] = true;
8         inTree[e.v][e.u] = true;
9     }
10
11     auto adj = g.getAdjMatrix();
12
13     for (int i = 0; i < n; i++) {
14         for (int j = i + 1; j < n; j++) {
15             if (adj[i][j] == 0 || inTree[i][j]) {
16                 continue;
17             }
18
19             auto path = pathInMST(i, j, mst);
20             if (path.empty()) {
21                 continue;
22             }
23
24             EdgeSet cycle;
25             for (int k = 0; k + 1 < (int)path.size(); k++) {
26                 int u = min(path[k], path[k + 1]);
27                 int v = max(path[k], path[k + 1]);
28                 cycle.insert({u, v});
29             }
30             cycle.insert({i, j});
31             basis.push_back(cycle);
32         }
33     }
34 }
35
36 EdgeSet CycleBasis::symDiff(const EdgeSet &a, const EdgeSet &b) {
37     EdgeSet result;
38     for (auto &e : a) {
39         if (!b.count(e)) {
40             result.insert(e);
41         }
42     }
43 }
```

```

41     }
42 }
43
44 for (auto &e : b) {
45     if (!a.count(e)) {
46         result.insert(e);
47     }
48 }
49 return result;
50 }

```

Listing 20: Построение фундаментального цикла и симметрическая разность (cycle_basis.cpp)

Метод `interactiveSymDiff()` предлагает пользователю выбрать номера циклов и последовательно вычисляет их симметрическую разность:

```

1 EdgeSet result = basis[indices[0]];
2 for (size_t i = 1; i < indices.size(); i++)
3     result = symDiff(result, basis[indices[i]]);

```

Listing 21: Симметрическая разность циклов (cycle_basis.cpp)

`symDiff(a, b)` возвращает рёбра, входящие ровно в одно из двух множеств.

Сложность: `compute()` Внешний двойной цикл перебирает все пары вершин — $O(V^2)$. Для каждой хорды вызывается `pathInMST` через BFS по матрице остова — $O(V^2)$. Итого:

$$O(V^2) \cdot O(V^2) = O(V^4).$$

`symDiff()` Два прохода по множествам рёбер размера не более V , каждый поиск в `set` стоит $O(\log V)$:

$$O(V \log V).$$

3. Результаты работы программы

При запуске программа выводит консольное меню (рис. 3), разбитое на пять разделов по лабораторным работам. Перед запуском любого алгоритма необходимо сгенерировать граф (пункт 1), выбрав число вершин, тип ориентированности и знак весов.

```
== Меню ==

ОБЩЕЕ
1. Сгенерировать граф
2. Вывести матрицу смежности
3. Вывести матрицу весов

ЛАБОРАТОРНАЯ РАБОТА № 1
4. Вычислить эксцентриситеты вершин
5. Найти центр графа
6. Найти диаметр графа
7. Матрица Шимбела
8. Найти все пути от А до В

ЛАБОРАТОРНАЯ РАБОТА № 2
9. Топологическая сортировка (алгоритм Фалкерсона)
10. Кратчайшие пути (алгоритм Флойда-Уоршалла)
11. Сравнить алгоритмы по числу итераций

ЛАБОРАТОРНАЯ РАБОТА № 3
12. Вывести матрицу пропускных способностей
13. Вывести матрицу стоимости
14. Максимальный поток
15. Поток минимальной стоимости [2/3 * max]

ЛАБОРАТОРНАЯ РАБОТА № 4
16. Число остовных деревьев (теорема Кирхгофа)
17. Минимальный остов (алгоритм Краскала)
18. Кодировать остов кодом Прюфера
19. Декодировать код Прюфера
20. Максимальное независимое множество вершин

ЛАБОРАТОРНАЯ РАБОТА № 5
21. Эйлеров цикл
22. Фундаментальная система циклов (MST + циклы)
23. Симметрическая разность циклов

0. Выход
Ваш выбор: _
```

Рис. 3: Главное меню программы

3.1. Генерация графа (пункты 1–3)

Пункт 1 запрашивает число вершин, тип ориентированности и знак весов, после чего генерирует граф по распределению Вейбулла. На рис. 4 и 5 показаны примеры генерации ориентированного и неориентированного графов. Пункты 2 и 3 выводят матрицы смежности и весов (рис. 6–9).

```

Введите количество вершин (>= 2): 5

Тип графа:
  1. Ориентированный
  2. Неориентированный
Ваш выбор [1/2]: 1

Тип весов рёбер:
  1. Только положительные
  2. Только отрицательные
  3. Смешанные
Ваш выбор [1-3]: 1

Граф сгенерирован (5 вершин).

```

Рис. 4: Генерация ориентированного графа (пункт 1)

```

Введите количество вершин (>= 2): 5

Тип графа:
  1. Ориентированный
  2. Неориентированный
Ваш выбор [1/2]: 2

Тип весов рёбер:
  1. Только положительные
  2. Только отрицательные
  3. Смешанные
Ваш выбор [1-3]: 1

Граф сгенерирован (5 вершин).

```

Рис. 5: Генерация неориентированного графа (пункт 1)

Матрица смежности:

	0	1	2	3	4
0	0	1	1	1	1
1	-	0	-	1	-
2	-	-	0	-	1
3	-	-	-	0	1
4	-	-	-	-	0

Рис. 6: Матрица смежности ориентированного графа (пункт 2)

Матрица весов:

	0	1	2	3	4
0	0	11	5	6	12
1	-	0	-	8	-
2	-	-	0	-	6
3	-	-	-	0	3
4	-	-	-	-	0

Рёбра графа:

```

0 -> 1 (вес: 11)
0 -> 2 (вес: 5)
0 -> 3 (вес: 6)
0 -> 4 (вес: 12)
1 -> 3 (вес: 8)
2 -> 4 (вес: 6)
3 -> 4 (вес: 3)

```

Рис. 7: Матрица весов ориентированного графа (пункт 3)

Матрица смежности:

	0	1	2	3	4
0	0	1	-	-	1
1	1	0	1	-	1
2	-	1	0	1	1
3	-	-	1	0	1
4	1	1	1	1	0

Рис. 8: Матрица смежности неориентированного графа (пункт 2)

Матрица весов:

	0	1	2	3	4
0	0	19	-	-	4
1	19	0	14	-	3
2	-	14	0	7	10
3	-	-	7	0	17
4	4	3	10	17	0

Рёбра графа:

- 0 -> 1 (вес: 19)
- 0 -> 4 (вес: 4)
- 1 -> 0 (вес: 19)
- 1 -> 2 (вес: 14)
- 1 -> 4 (вес: 3)
- 2 -> 1 (вес: 14)
- 2 -> 3 (вес: 7)
- 2 -> 4 (вес: 10)
- 3 -> 2 (вес: 7)
- 3 -> 4 (вес: 17)
- 4 -> 0 (вес: 4)
- 4 -> 1 (вес: 3)
- 4 -> 2 (вес: 10)
- 4 -> 3 (вес: 17)

Рис. 9: Матрица весов неориентированного графа (пункт 3)

3.2. Лабораторная работа 1 (пункты 4–8)

В первой лабораторной работе вычисляются метрики графа и ищутся пути. Пункт 4 (рис. 10) выводит эксцентриситет каждой вершины. Пункт 5 (рис. 11) находит центр графа и его радиус. Пункт 6 (рис. 12) выводит диаметр и диаметральные вершины вместе с путями между ними. Пункт 7 (рис. 13, 14) строит матрицу Шимбелла для минимальных и максимальных путей заданной длины k . Пункт 8 (рис. 15) перечисляет все простые пути между двумя заданными вершинами.

```
===== Эксцентриситеты вершин =====
Вершина 0: 1
Вершина 1: 2
Вершина 2: 1
Вершина 3: 1
Вершина 4: 0
```

Рис. 10: Пункт 4: эксцентриситеты вершин

```
===== Центр графа =====
Вершины центра: 4
Радиус графа: 0
```

Рис. 11: Пункт 5: центр графа и радиус

```

===== Диаметр графа =====
Диаметр: 2
Периферийные вершины: 0, 2

Периферийные пути:
0 -> 1 -> 2
2 -> 3 -> 4

```

Рис. 12: Пункт 6: диаметр графа и диаметральные вершины с путями

```

Тип матрицы Шимбела:
1. Минимальные пути
2. Максимальные пути
Ваш выбор [1/2]: 1
Длина пути К (от 0 до 4): 2

Матрица Шимбела (минимальные пути, K=2)

```

	0	1	2	3	4
0	0	-	-	19	9
1	-	0	-	-	11
2	-	-	0	-	-
3	-	-	-	0	-
4	-	-	-	-	0

Рис. 13: Пункт 7: матрица Шимбелла – минимальные пути

```

Тип матрицы Шимбела:
1. Минимальные пути
2. Максимальные пути
Ваш выбор [1/2]: 2
Длина пути К (от 0 до 4): 2

Матрица Шимбела (максимальные пути, K=2)

```

	0	1	2	3	4
0	0	-	-	19	11
1	-	0	-	-	11
2	-	-	0	-	-
3	-	-	-	0	-
4	-	-	-	-	0

Рис. 14: Пункт 7: матрица Шимбелла – максимальные пути

```

Начальная вершина A [0-4]: 0
Конечная вершина B [0-4]: 4

===== Все пути из 0 в 4 =====
Найдено путей: 4

1. 0 -> 1 -> 3 -> 4
2. 0 -> 2 -> 4
3. 0 -> 3 -> 4
4. 0 -> 4

```

Рис. 15: Пункт 8: все простые пути от вершины A до вершины B

3.3. Лабораторная работа 2 (пункты 9–11)

Во второй лабораторной работе реализованы топологическая сортировка и поиск кратчайших путей. Пункт 9 (рис. 16) выполняет топологическую сортировку алгоритмом Фалкерсона и выводит перенумерованные матрицы. Пункт 10 (рис. 17) вычисляет матрицу расстояний алгоритмом Флойда-Уоршалла и выводит кратчайший путь между двумя вершинами. Пункт 11 (рис. 18) сравнивает оба алгоритма по числу итераций.

```
== Алгоритм Фалкерсона ==

Группа 1: { 1 -> 0 }
Группа 2: { 2 -> 1 3 -> 2 }
Группа 3: { 4 -> 3 }
Группа 4: { 0 -> 4 }

Итоговая нумерация вершин:
Старый номер:  0  1  2  3  4
Новый номер:   4  0  1  2  3

Матрица смежности после нумерации Фалкерсона:

Матрица смежности:

    |      0      1      2      3      4
---+-----
0 |      0      1      1      1      1
1 |      -      0      -      1      -
2 |      -      -      0      1      1
3 |      -      -      -      0      1
4 |      -      -      -      -      0

Матрица весов после нумерации Фалкерсона:

Матрица весов:

    |      0      1      2      3      4
---+-----
0 |      0      8      7      2      7
1 |      -      0      -      -      -
2 |      -      -      0     10     21
3 |      -      -      -      0     19
4 |      -      -      -      -      0

Граф переупорядочен. Матрицы теперь отражают новую нумерацию.
```

Рис. 16: Пункт 9: топологическая сортировка алгоритмом Фалкерсона

== Алгоритм Флойда-Уоршалла ==

Матрица следующих вершин:

		0	1	2	3	4
	-----+					
0		0	1	2	3	4
1		-1	0	2	3	3
2		-1	-1	0	3	3
3		-1	-1	-1	0	4
4		-1	-1	-1	-1	0

Матрица расстояний (Фloyd-Уоршалл):

		0	1	2	3	4
	-----+					
0		0	11	11	4	7
1		-	0	10	7	29
2		-	-	0	10	32
3		-	-	-	0	22
4		-	-	-	-	0

Посмотреть конкретный путь?

1. Да

2. Нет

Ваш выбор [1/2]: 1

Начальная вершина [0-4]: 2

Конечная вершина [0-4]: 4

== Кратчайший путь: 2 -> 4 ==

Путь: 2 -> 3 -> 4

Длина: 32

Рис. 17: Пункт 10: матрица расстояний и кратчайший путь (Фloyd-Уоршалл)

```

== Сравнение алгоритмов по числу итераций ==
Вершин в графе: 5

Алгоритм Фалкерсона:      48 итераций
Алгоритм Флойда-Уоршалла: 125 итераций  (=  $n^3 = 5^3 = 125$ )

```

Рис. 18: Пункт 11: сравнение числа итераций алгоритмов Фалкерсона и Флойда-Уоршалла

3.4. Лабораторная работа 3 (пункты 12–15)

В третьей лабораторной работе реализована транспортная сеть с поиском максимального потока и потока минимальной стоимости. Пункты 12 и 13 (рис. 19, 20) выводят матрицы пропускных способностей и стоимостей. Пункт 14 (рис. 21) находит максимальный поток алгоритмом Форда–Фалкерсона. Пункт 15 (рис. 22–24) вычисляет поток минимальной стоимости величиной $\lfloor 2/3 \cdot F_{\max} \rfloor$.

Матрица пропускных способностей:

		0	1	2	3	4
0		0	11	11	4	7
1		-	0	10	7	-
2		-	-	0	10	-
3		-	-	-	0	22
4		-	-	-	-	0

Рис. 19: Пункт 12: матрица пропускных способностей

Матрица стоимостей:

		0	1	2	3	4
0		0	11	4	5	5
1		-	0	8	4	-
2		-	-	0	6	-
3		-	-	-	0	8
4		-	-	-	-	0

Стоимости рёбер:

- 0 -> 1 (стоимость: 11)
- 0 -> 2 (стоимость: 4)
- 0 -> 3 (стоимость: 5)
- 0 -> 4 (стоимость: 5)
- 1 -> 2 (стоимость: 8)
- 1 -> 3 (стоимость: 4)
- 2 -> 3 (стоимость: 6)
- 3 -> 4 (стоимость: 8)

Рис. 20: Пункт 13: матрица стоимостей рёбер

```

Текущий источник: 0,   сток: 4
Изменить? [1-да / 2-нет]: 2

Начальный остаточный граф (пропускные способности):

    |      0      1      2      3      4
----+-----
0 |      0      11      11      4      7
1 |      -       0      10      7      -
2 |      -       -       0      10      -
3 |      -       -       -       0      22
4 |      -       -       -       -       0

1. Найден увеличивающий путь: 0 => 4
   Узкое место (минимальная пропускная способность на пути): 7
2. Найден увеличивающий путь: 0 => 3 => 4
   Узкое место (минимальная пропускная способность на пути): 4
3. Найден увеличивающий путь: 0 => 1 => 3 => 4
   Узкое место (минимальная пропускная способность на пути): 7
4. Найден увеличивающий путь: 0 => 2 => 3 => 4
   Узкое место (минимальная пропускная способность на пути): 10

== Алгоритм Форда-Фалкерсона (Edmonds-Karp) ==
Источник: 0,   Сток: 4
Максимальный поток: 28

```

Рис. 21: Пункт 14: максимальный поток алгоритмом Форда–Фалкерсона

== Поток минимальной стоимости (Фloyd-Уоршалл) ==

Максимальный поток: 28, целевой [2/3]: 18
== Итерация 1 (пущено: 0 / 18) ==

Остаточные пропускные способности:

	0	1	2	3	4
0	0	11	11	4	7
1	-	0	10	7	-
2	-	-	0	10	-
3	-	-	-	0	22
4	-	-	-	-	0

Стоимости рёбер остаточной сети:

	0	1	2	3	4
0	0	11	4	5	5
1	-	0	8	4	-
2	-	-	0	6	-
3	-	-	-	0	8
4	-	-	-	-	0

Кратчайший путь: 0 => 4 (стоимость единицы потока: 5)

Узкое место: ребро 0 => 4 (пропускная способность: 7)

Пущено: 7 Итого поток: 7 Итого стоимость: 35

Рис. 22: Пункт 15: итерации поиска потока минимальной стоимости

```

== Итерация 2 (пущено: 7 / 18) ==

Остаточные пропускные способности:

  | 0 1 2 3 4
--+-----
0 | 0 11 11 4 -
1 | - 0 10 7 -
2 | - - 0 10 -
3 | - - - 0 22
4 | 7 - - - 0

Стоимости рёбер остаточной сети:

  | 0 1 2 3 4
--+-----
0 | 0 11 4 5 -
1 | - 0 8 4 -
2 | - - 0 6 -
3 | - - - 0 8
4 | -5 - - - 0

Кратчайший путь: 0 => 3 => 4 (стоимость единицы потока: 13)
Узкое место: ребро 0 => 3 (пропускная способность: 4)
Пущено: 4 Итого поток: 11 Итого стоимость: 87

```

Рис. 23: Пункт 15: продолжение итераций

```

== Итерация 3 (пущено: 11 / 18) ==

Остаточные пропускные способности:

  | 0 1 2 3 4
--+-----
0 | 0 11 11 - -
1 | - 0 10 7 -
2 | - - 0 10 -
3 | 4 - - 0 18
4 | 7 - - 4 0

Стоимости рёбер остаточной сети:

  | 0 1 2 3 4
--+-----
0 | 0 11 4 - -
1 | - 0 8 4 -
2 | - - 0 6 -
3 | -5 - - 0 8
4 | -5 - - -8 0

Кратчайший путь: 0 => 2 => 3 => 4 (стоимость единицы потока: 18)
Узкое место: ребро 0 => 2 (пропускная способность: 7)
Пущено: 7 Итого поток: 18 Итого стоимость: 213

== Итог ==
Источник: 0 Сток: 4
Целевой поток [2/3 * max]: 18
Достигнутый поток: 18
Минимальная стоимость: 213

```

Рис. 24: Пункт 15: итоговый поток и минимальная стоимость

3.5. Лабораторная работа 4 (пункты 16–20)

В четвёртой лабораторной работе выполняются операции с остовными деревьями и независимыми множествами. Пункт 16 (рис. 25) считает число остовных деревьев по теореме Кирхгофа. Пункт 17 (рис. 26) строит минимальный остов алгоритмом Краскала. Пункты 18 и 19 (рис. 27, 28) кодируют остов кодом Прюфера и декодируют его обратно с сохранением весов. Пункт 20 (рис. 29, 30) ищет максимальное независимое множество на исходном графе и на остове.

Матрица Лапласа:

	0	1	2	3	4
0	[2 -1 0 0 -1]				
1	[-1 3 -1 0 -1]				
2	[0 -1 3 -1 -1]				
3	[0 0 -1 2 -1]				
4	[-1 -1 -1 -1 4]				

Число остовных деревьев: 21

Рис. 25: Пункт 16: матрица Лапласа и число остовных деревьев (теорема Кирхгофа)

Рёбра минимального остова:

- 1 -- 4 (вес: 3)
- 0 -- 4 (вес: 4)
- 2 -- 3 (вес: 7)
- 2 -- 4 (вес: 10)

Суммарный вес: 24

Рис. 26: Пункт 17: минимальный остов алгоритмом Краскала

```

Исходный остов:
  1 -- 4 (вес: 3)
  0 -- 4 (вес: 4)
  2 -- 3 (вес: 7)
  2 -- 4 (вес: 10)
Суммарный вес: 24

Код Прюфера (4 элементов):
Последовательность: [ 4 4 2 4 ]
Веса рёбер:          [ 4 3 7 10 ]

```

Рис. 27: Пункт 18: кодирование остова кодом Прюфера

```

Код Прюфера (4 элементов):
Последовательность: [ 4 4 2 4 ]
Веса рёбер:          [ 4 3 7 10 ]

Декодированный остов:
  0 -- 4 (вес: 4)
  1 -- 4 (вес: 3)
  3 -- 2 (вес: 7)
  2 -- 4 (вес: 10)
Суммарный вес: 24

```

Рис. 28: Пункт 19: декодирование кода Прюфера

```

Выберите граф для MIS:
  1. Исходный граф
  2. Минимальный остов (MST)
Ваш выбор [1/2]: 1

Запуск на исходном графе...

Максимальное независимое множество (2 вершин):
Вершины: { 0 2 }

```

Рис. 29: Пункт 20: МНМ на исходном графе

```

Выберите граф для MIS:
  1. Исходный граф
  2. Минимальный остов (MST)
Ваш выбор [1/2]: 2

Запуск на минимальном остоле...

Максимальное независимое множество (3 вершин):
Вершины: { 0 1 2 }

```

Рис. 30: Пункт 20: МНМ на минимальном остоле

3.6. Лабораторная работа 5 (пункты 21–23)

В пятой лабораторной работе реализованы эйлеровы графы и система циклов. Пункт 21 (рис. 31) проверяет эйлеровость графа, при необходимости достраивает его и строит эйлеров цикл алгоритмом Хирхольцера. Пункт 22 (рис. 32) вычисляет фундаментальную систему циклов на основе минимального остова. Пункт 23 (рис. 33) вычисляет симметрическую разность выбранных фундаментальных циклов.

```

Граф не является эйлеровым. Модификация:
[шаг 1] нечётных вершин: 2 -> {1, 2}
  [-] удаляем 1 -- 2 (ребро есть, не мост)
Граф стал эйлеровым.

Эйлеров цикл (6 рёбер):
0 -> 4 -> 3 -> 2 -> 4 -> 1 -> 0

```

Рис. 31: Пункт 21: достройка до эйлерового графа и построение эйлерова цикла

Рёбра минимального остова:

1 -- 4 (вес: 3)

0 -- 4 (вес: 4)

2 -- 3 (вес: 7)

2 -- 4 (вес: 10)

Суммарный вес: 24

Фундаментальная система циклов (2 цикл(ов)):

[1] { 0-1 0-4 1-4 }

[2] { 2-3 2-4 3-4 }

Рис. 32: Пункт 22: фундаментальная система циклов на основе МОД

Фундаментальная система циклов (2 цикл(ов)):

[1] { 0-1 0-4 1-4 }

[2] { 2-3 2-4 3-4 }

Введите номера циклов через пробел:

> 1 2

Операция: $C1 \Delta C2$

Результат симметрической разности:

0 -- 1

0 -- 4

1 -- 4

2 -- 3

2 -- 4

3 -- 4

Рис. 33: Пункт 23: симметрическая разность фундаментальных циклов

Заключение

В ходе выполнения пяти лабораторных работ были реализованы все поставленные задания. В лабораторной работе 1 выполнена случайная генерация связанного ациклического ориентированного графа по распределению Вейбулла, вычислены эксцентриситеты вершин, выделены центр и диаметральные вершины, реализован метод Шимбелла для поиска кратчайших и самых длинных маршрутов заданной длины. В лабораторной работе 2 реализованы топологическая сортировка по алгоритму Фалкерсона, нахождение кратчайших путей алгоритмом Флойда-Уоршалла с выводом матрицы расстояний и сравнение алгоритмов по числу итераций. В лабораторной работе 3 построена сеть, найден максимальный поток алгоритмом Форда–Фалкерсона и вычислен поток минимальной стоимости величиной $\lfloor 2/3 \cdot F_{\max} \rfloor$. В лабораторной работе 4 подсчитано число остовных деревьев по теореме Кирхгофа, построен минимальный остов алгоритмом Краскала, выполнено кодирование и декодирование кода Прюфера с сохранением весов рёбер, а также найдено максимальное независимое множество вершин. В лабораторной работе 5 для неориентированного графа реализована проверка того, что граф является Эйлеровым, а также достройка графа до Эйлера на основе добавления и удаления рёбер. Построен эйлеров цикл алгоритмом Хирхольцера. На основе минимального остова (алгоритм Краскала) получена фундаментальная система циклов, и реализованна симметрическая разность которая для двух и более циклов позволяет строить новые циклы.

Достоинства реализации

Все пять лабораторных работ используют один и тот же сгенерированный граф, что помогает при тестировании работы разных алгоритмов на одних и тех же данных.

Тоже самое можно сказать про реализованные алгоритмы, которые можно переиспользовать в разных лабораторных работах. Например, алгоритм Краскала в четвертой лабораторной работе используется для построения минимального остова, а в пятой лабораторной работе для получения фундаментальной системы циклов. Это позволяет избежать дублирования кода.

Использование контейнеров STL позволяет не реализовывать базовые структуры данных вручную. Например, `vector` используется для хранения списков вершин и рёбер, `queue` и `stack` для обходов графа в BFS, `set` для хранения множеств рёбер при работе с фундаментальными циклами.

Недостатки реализации

Граф хранится в виде матрицы смежности, что влияет на использованную память и время поиска вершин. Это неэффективно для графов с маленьким количеством рёбер, так как все равно требуется $O(n^2)$ памяти. И при поиске соседей вершины приходится за $O(n)$ по всей строке матрицы.

В алгоритме Краскала приходится дублировать ребра в отдельную структуру данных, чтобы потом запускать DFS для проверки наличия циклов.

Нет ограничения на число вершин в графе, что может привести к ошибкам при генерации и дальнейшему падению программы.

Масштабирование

Можно заменить матричное представление графа на списки смежности, чтобы программа работала быстрее и занимала меньше памяти на разреженных графах.

Также можно добавить ограничение на число вершин в графе, чтобы избежать ошибок при генерации и падению программы.

И можно реализовать графический интерфейс для визуализации графа и результатов алгоритмов, что сделает программу более удобной и наглядной. Например через библиотеку Qt или с использованием веб-интерфейса на HTML/CSS/JS.

Список литературы

- [1] Секция «Телематика» [Электронный ресурс]. – URL: <https://tema.spbstu.ru/tgraph/> (дата обращения: 16.05.2026).
- [2] Новиков Ф. А. *Дискретная математика для программистов*. – 3-е изд. – СПб.: Питер Пресс, 2009. – 384 с.
- [3] Вадзинский Р. Н. *Справочник по вероятностным распределениям*. – СПб.: Наука, 2001. – 295 с.
- [4] Шапорев С.Д. ”Дискретная математика”. (дата обращения: 16.05.2026).
- [5] Эриксон Дж. Алгоритмы (дата обращения: 16.05.2026).